

Project Proposal: Autonomous Multi-Agent AI Research & Development Platform

Jerlshin JG

jerlshin.official008@gmail.com

May 21, 2026

Abstract

This proposal details the design and deployment steps for an Autonomous Multi-Agent AI Research & Development Platform. The system is designed as a Minimum Viable Product (MVP) tailored for a solo developer's portfolio, demonstrating mastery across Artificial Intelligence, MLOps, DevOps, Distributed Systems, and Full-Stack Engineering using 100% free and open-source, locally hostable technologies. *MVP*

1 Executive Summary

This project proposes the development of an **Autonomous Multi-Agent AI Research & Development Platform**. Inspired by advanced systems like Devin and Perplexity, it is a unified environment where specialized AI agents autonomously collaborate to research, write code, run machine learning experiments, and evaluate results.

Designed as a comprehensive MVP for a solo developer, this project serves as a masterclass portfolio piece. It simultaneously demonstrates proficiency across Full-Stack Engineering, AI/Agentic Workflows, MLOps, DevOps, and Distributed Systems, utilizing strictly free, open-source, and locally hostable technologies.

2 Core Objectives & Portfolio Value

local models → but compatible with openAI standards

Instead of building a fragmented portfolio of small applications, this single system demonstrates end-to-end software lifecycle mastery:

- **AI Engineering:** Local Large Language Model (LLM) orchestration (Ollama), Retrieval-Augmented Generation (RAG), tool calling, and multi-agent frameworks (LangGraph).
- **MLOps:** Automated experiment tracking, model evaluation, and deployment pipelines (MLflow).
- **Backend & Distributed Systems:** Asynchronous APIs (FastAPI), message queues (Redis), and vector search (Qdrant).
- **DevOps & CI/CD:** Containerization (Docker), local orchestration (k3s or Docker Compose), and automated pipelines (GitHub Actions).
- **Frontend:** Real-time streaming UI and dashboards (Next.js, WebSockets).

Portfolio ready Application.

3 Technology Stack (100% Free / Open Source)

Table 1: Selected MVP Tech Stack

Category	Technologies Chosen	Justification for MVP
Local AI Models	Ollama, Llama 3, Qwen, Mistral	Zero API costs; runs locally on consumer hardware.
Agent Framework	LangGraph, LangChain	Unmatched control flow for cyclical, multi-agent reasoning.
Backend & API	Python, FastAPI, Celery, WebSockets	Async-first, high performance, industry standard for AI.
Databases	PostgreSQL, Redis	Rock-solid, ubiquitous, easily containerized.
Vector Database	Qdrant or ChromaDB	Open-source, fast, excellent local Docker support for RAG.
MLOps	MLflow	Lightweight enough to run locally, powerful enough for tracking.
DevOps & Infra	Docker, Docker Compose, k3s	Allows one-click local deployment of the entire stack.
CI/CD	GitHub Actions	Free for public repositories.
Frontend	Next.js, TypeScript, Tailwind, shadcn/ui	Fast development, beautiful components, native API routes.
Observability	Prometheus, Grafana	Industry-standard open-source monitoring.

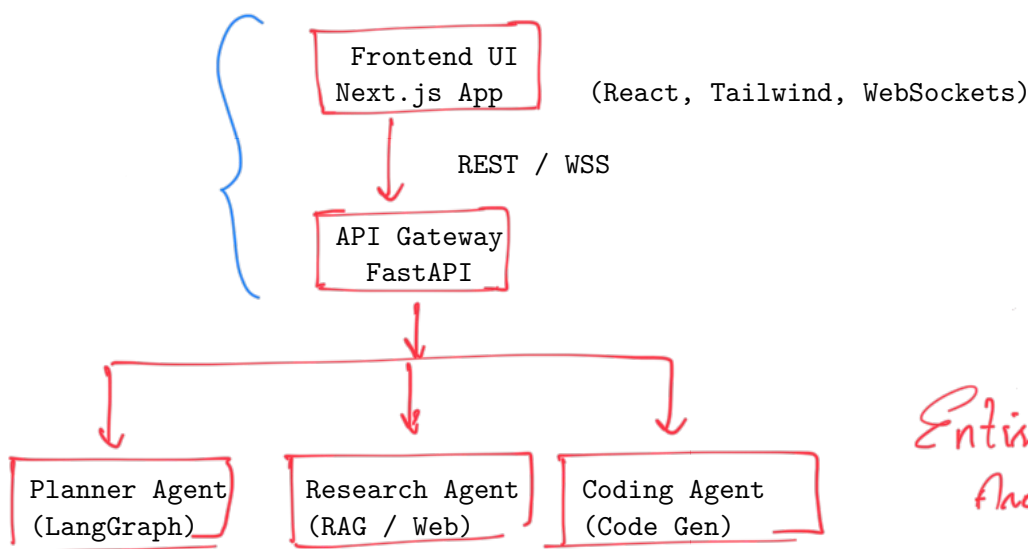
trying to running on multiple systems

learning all essential tools

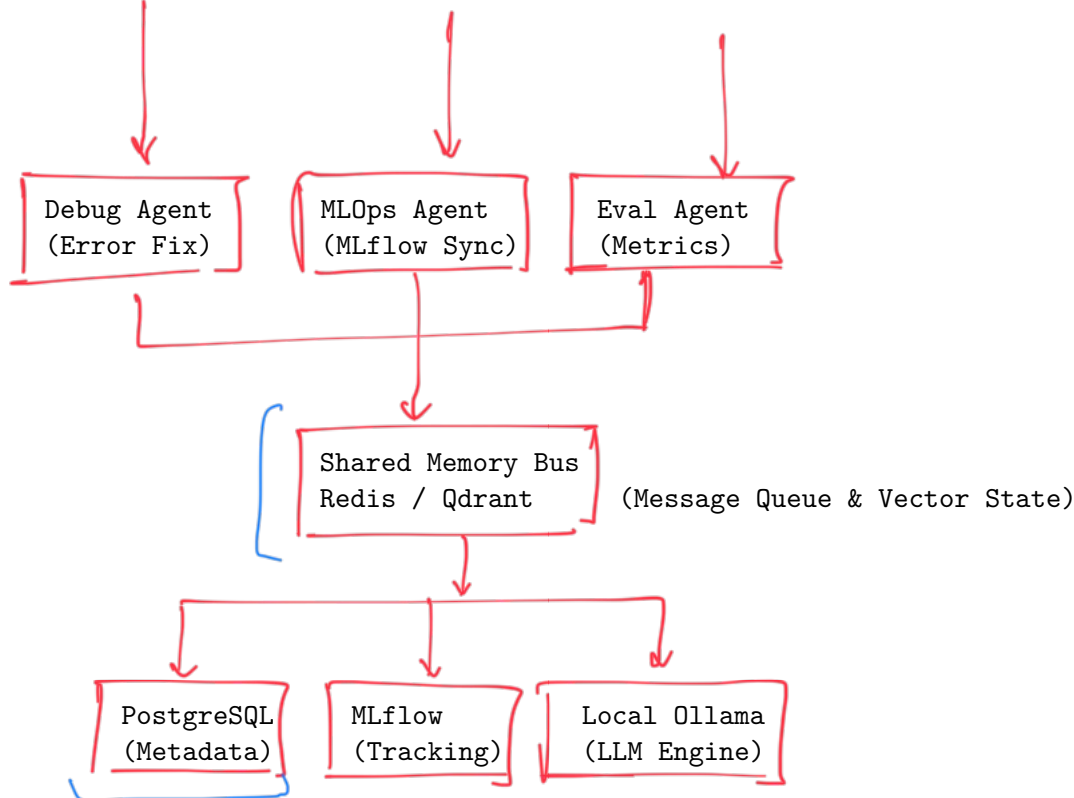
4 System Architecture

4.1 High-Level Component Diagram

The system follows a microservices-inspired architecture, communicating via REST and asynchronous message brokers.



Entire Application Architecture.



4.2 Multi-Agent Workflow *⇒ we will refine the agents later*

1. **Planner Agent:** Receives the user prompt, breaks it down into structured sub-tasks, and delegates them.
2. **Research Agent:** Searches the local vector database (RAG) or simulated web environments for necessary context.
3. **Coding Agent:** Writes Python/ML scripts based on research in an isolated Docker sandbox container.
4. **Debug Agent:** Reviews code, executes it, captures stderr/stdout, and fixes syntax or logic errors iteratively.
5. **MLOps Agent:** Executes the finalized training/experiment script and logs all metrics to MLflow.
6. **Eval Agent:** Checks MLflow for performance. If the model fails benchmarks, it triggers the Planner to attempt a new approach.

5 Development Phases & MVP Roadmap

To ensure successful completion by a solo developer, the project will be built in sequential, manageable phases.

5.1 Phase 1: Foundation & Infrastructure (Weeks 1-2)

- Initialize GitHub repository with a monorepo structure.
- Set up Docker Compose to orchestrate: PostgreSQL, Redis, Qdrant, MLflow, and Ollama.
- Create a basic FastAPI backend that connects successfully to all databases.

*we two → not too complex.
but, not too easy either*

Incremental advancement

1. Architecture first / Pipeline first.

5.2 Phase 2: Single Agent & RAG (Weeks 3-4)

- Implement a basic LangChain/LangGraph feedback loop.
- Build the Research Agent: document ingestion pipeline, chunking, embedding, and retrieval via Qdrant.
- Conduct baseline LLM reasoning tests using local Ollama (Llama 3/Qwen).

5.3 Phase 3: Multi-Agent Collaboration (Weeks 5-6)

- Implement the Planner, Coding, and Debug Agents.
- Create a safe, restricted Sandbox (a minimal, detached Docker container) where the Coding Agent can execute Python scripts without endangering the host system.
- Establish the LangGraph state machine to pass context and shared memory between agents.

Agents → we can decide later

5.4 Phase 4: MLOps Integration (Weeks 7-8)

- Implement the MLOps Agent.
- Create a workflow where agents write a simple ML script (e.g., scikit-learn classifier), execute it, and log the accuracy, parameters, and model artifacts directly to the MLflow container.

→ SIMPLE

5.5 Phase 5: Frontend & Observability (Weeks 9-10)

- Build the Next.js dashboard.
- Implement WebSockets to stream agent thoughts and sandbox console outputs to the user interface in real-time.
- Deploy Prometheus and Grafana via Docker Compose; configure them to scrape API latencies and Ollama token generation speed.

→ fullstack

6 GitHub Repository Structure

A clean, logical monorepo structure is critical to demonstrating architectural competence:

```
autonomous-ai-platform/  
  .github/  
    workflows/      # CI/CD pipelines (test, build, lint)  
  backend/  
    agents/         # FastAPI Application  
    api/            # LangGraph definitions (Planner, Coder, etc.)  
    core/           # REST endpoints and WebSockets  
    models/         # Config, DB connections, security  
    tools/          # Pydantic and SQLAlchemy models  
  frontend/        # Tools agents can use (Bash, search, MLflow)  
    components/     # Next.js Application  
    lib/            # React components (shadcn/ui)  
    pages/          # API clients, WebSocket hooks  
                   # Dashboard routes
```

```

infrastructure/      # DevOps & Deployments
  docker-compose.yml # Local one-click deploy
  k8s/              # (Optional) k3s manifests for later
  prometheus/       # Monitoring config
  grafana/          # Dashboard templates
mlops/              # MLflow & Sandbox environments
  scripts/          # Dummy datasets / baseline models
  sandbox.Dockerfile # Isolated container for agent execution
docs/               # Markdown documentation for submodules
.env.example
Makefile            # Shortcut tasks (e.g., make run, make test)
README.md

```

7 CI/CD Pipeline (GitHub Actions)

Since this is a solo-developer MVP, the pipeline focuses on Continuous Integration (Code Quality) and automated Docker verification, without the expense of a live cloud deployment.

7.1 Workflow Pipeline Steps (main.yml)

1. **Trigger:** Triggered automatically on push to the main branch or when a Pull Request is opened.
2. **Lint & Format:** Run Ruff for Python codebase and ESLint/Prettier for the Next.js application.
3. **Test:** Execute unit tests using pytest (backend) and Jest (frontend). LLM interactions are fully mocked to save compute resources and time.
4. **Build:** Verify the build processes by compiling Docker images for the Frontend and Backend services.
5. **Dry-Run Compose:** Boot up the complete stack with `docker-compose up -d` inside the GitHub runner to verify startup success, and gracefully tear down the environment.

8 Observability & Benchmarks

8.1 Observability (Prometheus + Grafana)

- **System Metrics:** Active CPU, memory usage, and health states of all running Docker containers.
- **LLM Performance:** Tokens per second, time-to-first-token (TTFT), and GPU usage (if utilizing PyTorch with ~~CUDA/ROCm~~ locally). *Metal (Silicon)*
- **API Performance:** HTTP endpoint latency, error rates (500-range codes), and WebSocket connection counts.

8.2 Agent Benchmarking Framework (The "Eval Agent")

- **Task Success Rate:** Percentage of auto-generated tasks that execute without syntax or run-time exceptions.

- **Debugging Iterations:** Count of self-correction loops between the Coding and Debug Agents before code succeeds.
- **RAG Precision:** Subjective relevance scoring of context documents evaluated by a local evaluation-tuned model.

9 Documentation Strategy

Thorough documentation sets a major portfolio project apart from smaller hobby apps.

- **Root README.md:** Features an architecture overview, a brief video demo/GIF link, and a fast "Quickstart" block using Docker Compose.
- **Submodule Docs (/docs):**
 - AGENTS.md: Details state routing within LangGraph, prompt designs, and system instructions.
 - MLOPS.md: Explains the MLflow tracking pipelines and metadata schemas.
 - ARCHITECTURE.md: Details interactions between Redis, PostgreSQL, and Qdrant.
- **Inline Documentation:** Standardized Python type hinting, docstrings, and cleanly typed TypeScript interfaces.

We don't need production ready software → just an MVP
 & exp
 +
 portfolio